

RESEARCH ARTICLE

Efficient Prefetch and Issue Scheduling Approaches for Simultaneous Multithreading Applied to Superscalar RISC-V Processor

HANANYA RIBO¹ AND SHLOMO GREENBERG^{1,2}, (Member, IEEE)¹School of Electrical and Computer Engineering, Ben-Gurion University of the Negev, Beer-Sheva 8410501, Israel²Computer Science Department, Sami Shamoon College of Engineering, Beer-Sheva 8410802, Israel

Corresponding author: Shlomo Greenberg (shlomgr@sce.ac.il)

ABSTRACT Thread level parallelism (TLP) is a common approach to achieve parallelism where Instruction level parallelism (ILP) is insufficient. Hardware multithreading is a prevalent approach in the micro-architecture layer for tolerating long events such as memory access, mispredictions, and accelerators latencies by utilizing idle cycles and avoiding CPU stalling. Multithreading architectures are commonly used across many processors and various embedded edge devices to improve performance. This work suggests a new 2-way superscalar in-order Multithreading (MT) micro-architecture applied to a superscalar RISC-V processor. This work also proposes efficient prefetch-scheduling and Issue-scheduling algorithms for Simultaneous Multithreading (SMT) processors. The suggested micro-architecture was implemented on an FPGA chip. The results show that the proposed prefetch-scheduling and issue-scheduling algorithms outperform the 2-way superscalar Fine-Grained Multi-Threaded (FGMT) and the Coarse-Grained Multi-Threaded (CGMT) approaches. The proposed architecture is evaluated using the standard MiBench and RISC-V benchmarks, demonstrating an average processor core utilization improvement of up to 52% in terms of IPC using four threads.

INDEX TERMS Hardware multithreading, prefetch scheduling, issue scheduling, superscalar, RISC-V, in-order pipeline.

I. INTRODUCTION

A scalar single-thread pipeline can exploit Instruction Level Parallelism (ILP) in case of its existence in the code or in case of data-forwarding-based micro-architecture, otherwise CPU stalls are inevitable. In addition to data dependence that reduces ILP there are two more factors named control and structural dependencies that cause stalls. However, by utilizing CPU stalls we create a distance between dependent instructions, and as a result, the dependence is cancelled but at a price of CPU performance degradation. In a 2-Way superscalar single-thread pipeline, the ILP constraints are defined in two dimensions. The first dimension is in a way of pipeline route, namely horizontal direction. The second dimension is between two instructions in each pipeline stage, namely

vertical direction. In a superscalar single-thread pipeline, the case of data dependence in a shared pipeline stage cannot be handled using data forwarding and hence CPU stalls are inevitable. TLP is a common approach to achieve parallelism where ILP is insufficient. Hardware multithreading is a common and efficient strategy used to exploit TLP and Data Level Parallelism (DLP) to improve single-thread pipeline utilization.

In general-purpose superscalar single-thread Out-of-Order (OoO) CPUs [1], [2], multiple instructions are issued in a single clock cycle and are processed in parallel, and the program order of the instructions can be swapped. Out-of-Order CPUs have been implemented in many processors to improve their performance. However, actual programs often lack sufficient ILP, which makes parallel execution difficult or impossible. Therefore, the use of ILP in a single-instruction flow is limited. In contrast, because no dependencies exist

The associate editor coordinating the review of this manuscript and approving it for publication was Liang-Bi Chen.

between different threads (except for synchronous processing) a parallelism can be extracted more easily with multiple instruction flows than a single instruction one. Multithreaded architectures that focus on TLP have been proposed as an efficient approach to improve processor performance and implemented in actual processors [3], [4], [5]. Out-of-order execution may not be cost-effective when applied to processors featuring simultaneous multithreading (SMT) [6].

Multithreading allows multiple threads to share the functional units of a single processor core in an overlapping fashion by duplicating the context of each thread, i.e., using a separate instruction queue (IQ), register file (RF) and program counter (PC) for each thread.

Hardware multithreading can significantly increase processor throughput compared to single-threaded-based cores by hiding idle cycles and avoiding CPU stalls such as memory latencies, branch mispredictions, and accelerators latencies. Multithreading is supported in various processor architectures. The IBM Power9 processor [7] has an enhanced core architecture optimizing for emerging workloads with superior thread performance. It supports up to eight hardware threads built on the same basic building block of the Power9 core micro-architecture. The IBM z13 introduces high instruction execution parallelism using a Simultaneous Multithreading (SMT) architecture that supports concurrent execution of two threads [8].

Among the main traditional scheduling approaches are the FGMT, CGMT, and SMT [9]. The FGMT, also called Interleaving Multithreading (IMT) [9], [10], is a multithreading scheduling technique that issues an instruction from a different hardware thread at every clock cycle using a round-robin scheduler [9], [11]. CGMT scheduling algorithm suggests adapting a switch-on-event technique forcing switching to the next thread in case of a high latency event such as a cache miss [12]. SMT scheduling algorithm, also called Hyper-threading, is a multithreading scheduling technique that supports multi-issue, i.e., issuing multiple instructions from different threads simultaneously [9].

Interleaved multithreaded architectures have been proven as an advantageous solution to maximize pipeline utilization when it comes to executing parallel applications since different threads can operate different instruction processing phases in the same cycle. Cheikh et al. [13] have expanded the target applications of IMT micro-architecture to support mathematical and convolution operations efficiently. Benchara et al. [10] present an in-order five-stage pipeline interleaved multi-threaded processor with two hardware thread contexts for embedded systems. The interleaved multithreading core has an augmentation of 73.2% in the core area, while the average performance gain is 17% compared to the single-threaded core [10].

Switch-on-event multithreading (also known as coarse-grained MT or block MT) processors run multiple threads on a pipeline machine, while the pipeline switches threads on stall events (e.g., a cache miss). The thread switch penalty is determined by the number of stages in the pipeline that

are flushed. Kvatinisky et al. [14] present a continuous flow multithreading using a dedicated multi-state pipeline register to hold the state of multiple threads within the execution pipeline stages, where only one thread is active at a time.

The SMT approach allows the simultaneous issuing of multiple instructions from multiple threads on each cycle [8], [15]. SMT requires complex architectures involving dynamic scheduling hardware. Arithmetic units and resources other than those specific to each thread can be shared among threads, enabling multithreaded execution with fewer resources than a simple multicore system. Several models of SMT techniques have been presented [15]. Results show that SMT has the potential to achieve four times the throughput of superscalar and double that of fine-grained multithreading. However, while SMT increases processor utilization, it adds substantial complexity to the design [15].

This work suggests a new micro-architecture design for a 2-way superscalar RISC-V SMT in-order CPU. We propose efficient new prefetch and Issue scheduling algorithms for Simultaneous Multithreading that improve the CPU core utilization relative to 2-way superscalar RISC-V CPUs. The suggested superscalar SMT micro-architecture supporting the RISC-V ISA was fully implemented in Verilog, verified, and realized on FPGA chip. The proposed prefetch and issue scheduling algorithms have been applied to RISC-V ISA and demonstrate decoding of the instruction type in an early stage of the pipeline. Results show that the proposed prefetch and issue scheduling algorithms outperform the RR, and FGMT, CGMT algorithms respectively. The proposed architecture is evaluated using the Mi-Bench [16] and RISC-V [17] benchmarks, demonstrating an average processor utilization improvement of 52% in terms of IPC using four HW threads. The rest of the paper is organized as follows. Section II presents the related work. Section III introduces the proposed RISC-V, 2-way superscalar SMT processor core architecture. Section IV introduces the proposed prefetch scheduling algorithm and its implementation. Section V introduces the proposed issue scheduling algorithm and its implementation. Section VI demonstrates the experimental results, and analyzes the results in terms of Power, Performance, and Area (PPA) of the suggested processor core. Section VII summarizes the paper.

II. RELATED WORK

RISC-V, a new open-source instruction set architecture (ISA), developed at the University of California, Berkeley (UCB) in 2010, has attracted considerable attention recently. RISC-V suggests an efficient and cheap alternative to the well-known ARM and X-86 architectures. This innovative approach paved the way for computer architects to design efficient strong cores supporting complex instruction set extensions [18]. The RISC-V ISA is becoming an increasingly popular ecosystem for embedded hardware and software development and is widely used in industry and academia. Asanovic et al. [19] describe the Berkeley Rocket SoC open-source design generator used to design

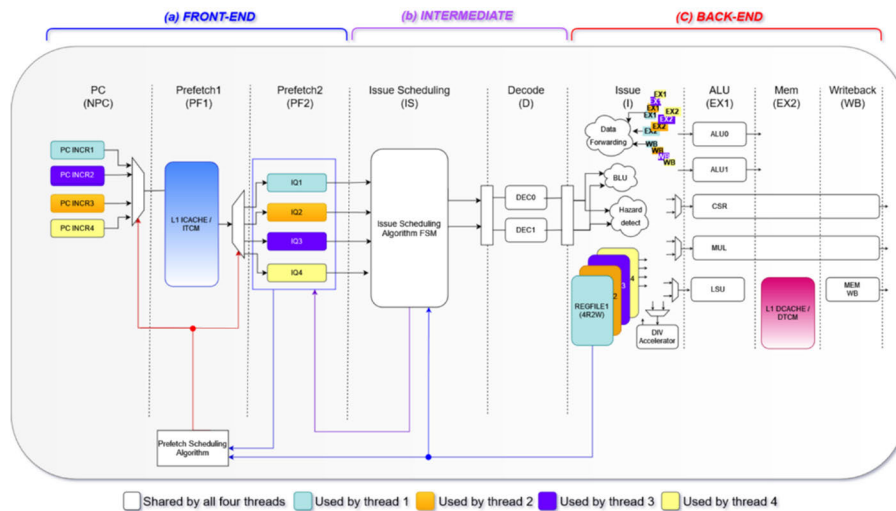


FIGURE 1. SUMT-V micro-architecture, 2-way superscalar multi-threaded in-order CPU.

synthesizable general-purpose cores composed of RISC-V ISA and supports both in-order (Rocket) and out-of-order cores (BOOM) [1], [19]. Various multicore platforms with clusters of RISC-V cores have been proposed as an alternative model for real-time applications that consume high computing power [20], [21]. Schiavone et al. [21] introduce low-power RISC-V cores targeting IoT applications and control-oriented tasks. Conti et al. [20] propose the PULP (Parallel Processing Ultra-Low Power platform), an architecture built on clusters of tightly coupled RISC ISA cores, using advanced techniques for fast performance and energy scalability. Patsidis et al. [22] present a synthesizable RISC-V dual-issue processor leveraging the compressed instruction set extension, eliminating unnecessary CPU stalls due to dependencies. The *Riscy* and the *Ultra-embedded* repositories [23], [24] provide a collection of open-source RISC-V processors that can be used in different simulation frameworks or FPGA hardware platforms.

Various RISC-V-based hardware multi-threaded architectures have been proposed supporting multi-threaded execution in a single core [25], [26], [27], [28]. Collange et al. [25] present the *Simty* multi-threaded RISC-V-based architecture implementing a SIMT execution model purely at the micro-architecture level. Cheikh et al. [26] present a microarchitecture of a multi-threaded RISC-V compliant processing core family for IoT, implementing interleaved multithreading. Mendoza et al. [27] describe the design and implementation of a multi-threaded RISC-V core based on the *Lagarto* processor architecture. Eni et al. [28] present a multi-threaded expansion of *RISCY* scalar micro-architecture and an Efficient Hint-based Event (EHE) Issue Scheduling algorithm for performance improvement.

III. THE PROPOSED SUMT-V PROCESSOR ARCHITECTURE

This section describes in detail the proposed Superscalar Multi-Threaded RISC-V (SUMT-V) micro-architecture

applied to RISC-V ISA. The proposed SUMT-V architecture is comprised of nine pipeline stages and is subdivided into three main parts: (a) FRONT-END, (b) INTERMEDIATE, and (c) BACK-END. The original RISC-V ISA, the various parts and pipeline stages are elaborated in the following sections.

A. RISC-V ISA

The RISC-V ISA supports both 32-bit and 64-bit register files. The ISA user specification [29] defines the following various RISC-V ISA extensions: (a) *I-Integer* RV32I (or RV64I) is the basic instruction set that includes the minimum instructions required for computer operation, such as integer arithmetic, branch, and jump instructions. (b) *M-extension*, extended instructions such as integer multiplication/division. (c) *A-Atomic* extension for Memory Operations. (d) *F-D-Q* extensions supporting Single-Double and Quad Precision Floating Point. (e) *V-extension*, extended vector instructions that suit embedded systems. The RISC-V privilege architecture [30] includes the following three ISA modules: Machine ISA (M-mode), Supervisor ISA, and Hypervisor ISA. This work suggests supporting the RV32-IM, i.e., in 32-bit Integer with multiplication and division extension, and the privileged M mode.

B. SUMT-V MICRO-ARCHITECTURE

Fig.1 depicts the proposed SUMT-V core architecture, based on a two-way superscalar SMT micro-architecture. The proposed 2-way core pipeline supports in-order flow and is composed of the following 9-stages: (1) Next PC (NPC), (2) Instruction Prefetch stage 1 (PF1), (3) Instruction Prefetch stage 2 (PF2), (4) Issue Scheduling (IS), (5) Decode (D), (6) Issue (I), (7) Execution stage-1 (EX1), (8) Execution stage-2 (EX2), and (9) Write-Back (WB). The proposed pipeline architecture can be subdivided into three main parts: (a) FRONT-END, (b) INTERMEDIATE, and (c) BACK-END.

C. FRONT-END

As depicted in Fig.1(a), the FRONT-END pipeline section includes the first three stages in the proposed pipeline: (1) NPC - Next PC, (2) PF1- Instruction prefetch stage-1, and (3) PF2 - Instruction prefetch stage-2. The front-end unit is responsible for calculating the next Program Counter (PC) for each thread, and prefetching instructions from memory (i.e., from L1 I-Cache/ITCM) into each dedicated thread Instruction Queue (IQ).

To support hardware multithreading, each thread is assigned a unique hardware context, including (a) Thread ID (TID), (b) a dedicated PC register, (c) an instruction queue (ThreadIQ), (d) a unique register file (RF), and (e) a specific forwarding data path. The instruction queue of each thread (ThreadIQ) is composed of a FIFO-based structure at a depth of eight 32-bit instruction words. Each entry in the ThreadIQ contains the following information: two 32-bit instruction words, Thread PC, thread TID and two System Status bits. This information content flows through the pipeline and is always associated with a specific TID.

The PF1 stage is responsible for prefetching a double word (i.e., two adjacent instructions) on each clock cycle, from L1 I-Cache/ITCM to the ThreadIQ according to the specific Program Counter (PC) which is associated with the current TID. The unique prefetch-scheduling algorithm is described in detail in Sec. IV. In the PF2 stage, eight instructions (two from each ThreadIQ) are routed and saved into the *IIS-BUF* instructions IS buffer (i.e. *PF2-IS* stage buffer). Ideally, the PF2 stage should provide a continuous stream of eight instructions to the IS stage. Each IQ consists of a FIFO structure containing eight instructions, organized in double word width and depth of four. Initially, while all four IQs are empty, it takes 16 cycles to fill up all the queues since the core prefetches a double word for each cycle. While the issue scheduling algorithm can route, in the PF2 stage, two instructions (double word) from any available Queue each cycle, no stall is involved, even if some of the queues are empty. Since filling the queues can be carried out during pulling instructions from the queues, there is no latency involved in filling the queues unless the issue scheduler requires access to an empty queue, which causes a cycle stall. Moreover, any stall occurrence at the Issue stage can be utilized to fill out the IQs if needed.

D. INTERMEDIATE

Fig.1(b) depicts the two intermediate stages: Issue Scheduling (IS) and Decode (D). The intermediate unit is responsible for issuing a stream of two instructions from the threads' instruction queues output (which contains eight instructions) into the Decoding stage. Fig.2 depicts the IS stage input of the available four Instruction Queues output, i.e., four double words of *IIS-BUF* size of 2×4 instructions. The issue scheduler output is a continuous stream of instruction pairs (i.e., two 32-bit words) from *IIS-BUF* ideally on each cycle clock, till all instructions are passed on to the decode stage.

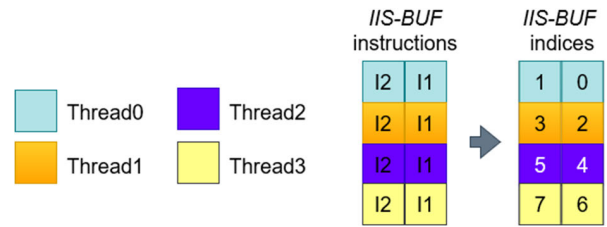


FIGURE 2. Output of the four hardware threads' Instruction Queues represented in *IIS-BUF* size of 2×4 . *IIS-BUF* indices 0,1 are the IQ0's output channels 1,2 respectively. *IIS-BUF* indices 2,3 are the IQ1's output channels 1,2 respectively. *IIS-BUF* indices 4,5 are the IQ2's output channels 1,2 respectively. *IIS-BUF* indices 6,7 are the IQ3's output channels 1,2 respectively.

Ideally, the issue scheduling algorithm is to exploit the TLP among the threads' instructions by selecting the ideal stream of instruction pairs. The selection is under consideration of structural, data, and control dependencies. In the case of an in-order machine with four hardware threads. The permutations of choosing four instruction pairs that cover a given *IIS-BUF* is 52 [31]. This number grows significantly with the growth of hardware threads, as a result direct approach of full permutation coverage-based algorithm is infeasible. This work suggests an efficient low-cost SMT Issue scheduling algorithm and its implementation that will be described in Section V.

E. BACK-END

Fig.1(c) shows the last four pipelines' stages: (1) Issue stage (I), (2) Execution stage-1 (EX1), (3) Execution stage-2 (EX2), and (4) Write-Back stage (WB). The back-end unit is responsible for issuing two decoded instructions, in each cycle, to the appropriate functional units for execution and writeback operation. The Issue stage is composed of five sub modules: (1) RF module which includes 4 register files, one for each hardware thread. (2) Hazard detection unit based on score boarding logic for detecting data dependencies, (3) Data forwarding of the four output Functional Units (FUs), (4) Branch Logic Unit (BLU), that is responsible for branch address resolution and misprediction signaling, (5) Division accelerator. The last three pipeline stages include two execution stages EX1, EX2 and the write-back WB stage.

The data forwarding is comprised of four different routes as the number of threads, derived from different origins: (1) ALU1, ALU2 outputs in EX1 stage, (2) MUL output in EX2 stage, (3) Load memory in EX2 stage, and (4) Write-Back stage. The execution unit is composed of five functional units: (1) ALU1 and (2) ALU2. The two ALUs for each pipeline are used for simple arithmetic and logic operations. The results of the ALUs are ready after one cycle (in EX1 stage). Using two ALUs is valuable due to the prevalence of arithmetic instructions. (3) A single multiplier module (MUL) is assigned only to channel-1 (out of the 2-way). The result of the multiplication is ready after two cycles (in EX2 stage). (4) Load/Store Unit (LSU), and (5) CSR unit that implements dedicated CSR instructions. Table 1 depicts the

execution latency for the five functional units, and the stage from which the forwarding is performed. The execution order of instructions belonging to the same thread is in-order, while instructions fetched from different threads can be executed out-of-order. The proposed execution pipeline can be easily extended to support other ISA extensions (like floating-point instructions).

TABLE 1. The execution (EX) latencies and forwarding distance of different instruction types.

Functional Unit type	Latency (cycles)	Forwarding stage
ALU1, ALU2 (pipelined)	1	EX1
Multiplication (pipelined)	2	EX2
Division (non-pipelined)	32	None
Load/Store (pipelined)	2	EX2

IV. THE PROPOSED PREFETCH SCHEDULING APPROACH

The prefetch algorithm, so called IQs Load Balancing (IQLB), is responsible for dynamic load balancing of the four thread IQs. The main purpose is to provide a continuous stream of eight instructions (i.e., four double words) in parallel to the issue scheduler. Since the issue scheduler empties the thread IQs by issuing instructions from the queues, the prefetch scheduler should adaptively fill the appropriate queues. Therefore, on average the total number of prefetched instructions is equally distributed among the four thread IQs.

A. IQLB ALGORITHM

Fig.3 depicts the IQLB algorithm structure composed of high-level priority decision with static and dynamic sub-hierarchies. The static sub-hierarchy is chosen where at least one of thread IQs is empty, otherwise dynamic sub-hierarchy is chosen unless all the thread IQs are full and hence the current state is preserved. The static sub-hierarchy is comprised from priority decision of four levels, the first level is TID0, and the last level is TID3. Each level's condition is true if the thread is valid, and its IQ is empty. The dynamic sub-hierarchy is comprised from priority decision of four levels, the first level is the TID of the thread IQ with the least number of instructions and the last level is the TID of the thread IQ with the greatest number of instructions. Each level's condition is true if its thread is valid, its thread IQ is not full, and its thread has no chance of branch misprediction that could lead to IQ flushing.

B. IQLB MODULE

Fig.5 depicts the IQLB algorithm module ports which includes five inputs, and one output (Number of threads = 4) as follows. (1) *thread_idx_valid*[3:0], each input bit is associated with a thread's Register File (RF) bit RF[tp]<31> which indicates whether a hardware thread is valid or not. Generally, a thread is not valid in two cases, first, on its finish of code running. Second, where a thread is not activated from the SW level. (2) *iq_idx_empty*[3:0], each input bit is associated with an IQ and indicates whether an IQ is empty or not. (3) *iq_idx_accept*[3:0], each input bit is

associated with an IQ and indicates whether an IQ is full or not. (4) *thread_idx_sorted* [3:0][1:0], each input entry is associated with a TID value of in a sorted TIDs series, by their IQ counter value (i.e., IQ's available instructions). Fig.6 shows the logic of the TID sorting network according IQs' counter value, an example marked in red color. (5) *decode_idx_is_branch*[3:0], each input bit is associated with a thread and produced by "OR" logic between two indicator bits forwarding from the decode stage. This bit indicates whether at least one of the current instructions in the decode stage (width of two) is branch or not. (6) *prefetch_tid*[1:0] is the algorithm output presents the chosen TID for prefetching, this value is evaluated on each cycle. IQLB module port (1) is logic stable where the rest are clock synchronized.

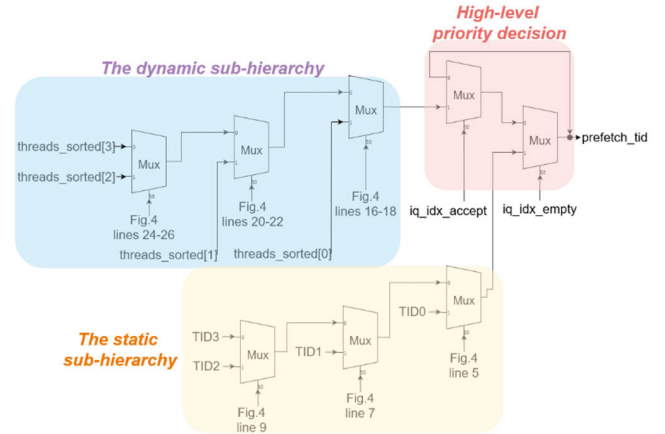


FIGURE 3. IQLB prefetch scheduling algorithm shown as a structure of high-level priority decision with static and dynamic sub-hierarchies.

```

1 always @*
2 begin
3   if(iq_idx_empty) // at least one IQi is empty
4     begin
5       if(iq_idx_empty[0] & thread_idx_valid[0])
6         prefetch_tid = `TID0; // threadID=0(master thread)
7       else if(iq_idx_empty[1] & thread_idx_valid[1])
8         prefetch_tid = `TID1; // threadID=1
9       else if(iq_idx_empty[2] & thread_idx_valid[2])
10        prefetch_tid = `TID2; // threadID=2
11      else
12        prefetch_tid = `TID3; // threadID=3
13    end
14  else if(iq_idx_accept) // at least one IQi is not full
15    begin
16      if(iq_idx_accept[thread_idx_sorted[0]] &
17         thread_idx_valid[thread_idx_sorted[0]] &
18         !decode_idx_is_branch[0])
19        prefetch_tid = thread_idx_sorted[0];
20      else if(iq_idx_accept[thread_idx_sorted[1]] &
21              thread_idx_valid[thread_idx_sorted[1]] &
22              !decode_idx_is_branch[1])
23        prefetch_tid = thread_idx_sorted[1];
24      else if(iq_idx_accept[thread_idx_sorted[2]] &
25              thread_idx_valid[thread_idx_sorted[2]] &
26              !decode_idx_is_branch[2])
27        prefetch_tid = thread_idx_sorted[2];
28      else
29        prefetch_tid = thread_idx_sorted[3];
30    end
31 end

```

FIGURE 4. IQLB HDL pseudo-code description in structure of combinatorial process.

C. IQLB IMPLEMENTATION

Fig.4 depicts the IQLB combinatorial implementation as an HDL pseudo-code. The circuit is composed of two high-level

decision priorities where each of them is comprised of four decision sub-hierarchies. The synthesis of these two main decision hierarchies is based on a cascade of two priority MUXs. Line 3 depicts the first-priority MUX with control condition of emptiness of an IQ. Lines 5-13 depict the internal hierarchy of the main first-priority. This internal hierarchy is a cascade of four priority MUXs with control condition of IQ_i emptiness and thread TID_i validity, and static input of an TID_i where 0 is the highest priority up to 3, the lowest priority. In case there is not an empty IQ and there is at least a non-full IQ, the second main hierarchy is selected. Line 14 depicts the second priority MUX with control condition of at least a non-full IQ. Lines 15-30 depict the internal hierarchy of the main second priority. This internal hierarchy is a cascade of four priority MUXs with control condition composed of three parts depicted in Fig.5 as IQLB module inputs: (1) The sorted TID values represented by *thread_idx_sorted*. Index 0 is the highest priority control condition containing the IQ_i with the minimum instructions up to index 3, the lowest priority control condition containing the IQ_i with maximum instructions. (2) The validity of each thread is indicated by *thread_idx_valid*. (3) The indication *decode_idx_is_branch* for each thread that branch instruction isn't decoded at the decode stage. The idea is to prevent a possible needless load of an IQ_i due to a chance of branch misprediction that could lead to IQ_i flushing.

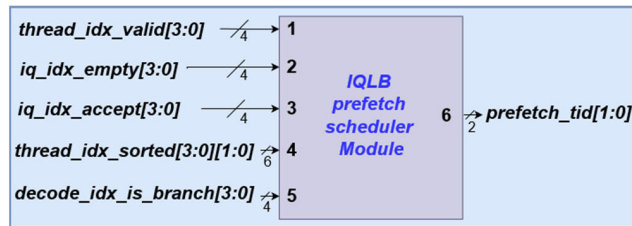


FIGURE 5. IQLB prefetch scheduler module includes ports of five inputs and one output.

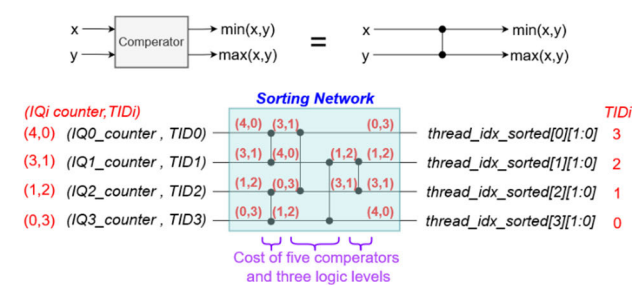


FIGURE 6. The shown digital circuit sorts four ordered pairs by their first parameter where each pair is composed of two parameters. The first parameter is the number of available instructions in an IQ_i. The second parameter is the IQ_i number, that is, the TID_i. See example written with red color.

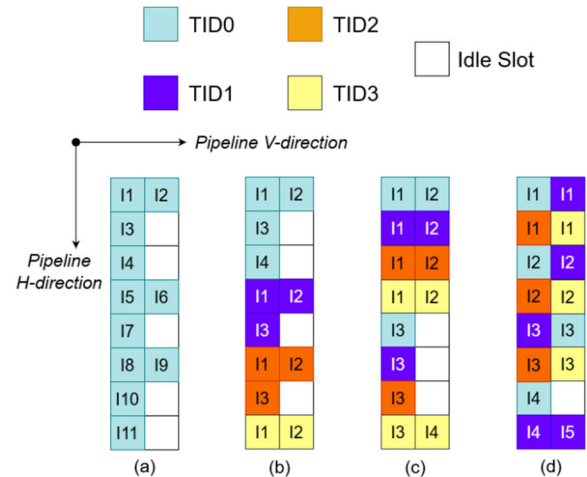
V. THE PROPOSED ISSUE SCHEDULING ALGORITHM

This section describes in detail the proposed SMT issue scheduling algorithm and its implementation. The efficiency of the proposed algorithm is compared to three main

traditional scheduling approaches (ST, CGMT, and FGMT). The following section presents a brief introduction to the existing scheduling policies.

A. ISSUE SCHEDULING POLICIES

Fig.7 depicts visually the main idea of the following four issue scheduling policies for 2-way in-order pipelines: (a) Single Thread Scheduling (ST), (b) Coarse-Grained Multi-Threaded (CGMT), (c) FGMT, and (d) Simultaneous multithreading (SMT) [9]. Fig. 7a depicts the ST scheduling, since it is a single thread, in case of data dependence between two adjacent instructions it is impossible to exploit the 2-way and issue them in parallel. Fig. 7b depicts the CGMT scheduling, the algorithm issues instructions of the same thread and switches to the next thread (RR fashion) only on long latency event, i.e., high latency operation such as cache miss, IQ flush due to branch misprediction, divider business (latency of 32 cycles), etc. In case of data dependence between two adjacent instructions of the same thread it is impossible to exploit the 2-way and issue them in parallel. Fig. 7c depicts the FGMT scheduling; the algorithm issues two instructions of the same thread in parallel on each cycle under thread policy switching of RR kind. In case of data dependence between two adjacent instructions of the same thread only one instruction is issued instead of two, without exploiting the 2-way. Fig. 7d depicts the SMT scheduling; the algorithm issues two instructions of the same thread or different threads in parallel on each cycle where thread policy switching is completely flexible.



Code example Dual-Issued of different scheduling policies
(a) Single Thread, (b) CGMT, (c) FGMT, (d) SMT

FIGURE 7. Four different issue scheduling policies.

In contrast to CGMT and FGMT, the SMT has full freedom of issue scheduling. In case of data dependence between two adjacent instructions of the same thread in an any cycle, we can issue two instructions of different threads. Therefore, SMT can be utilized better to hide dependencies of almost any kind, at the cost of algorithm complexity. Note that in all

MT issue scheduling, the thread who is in long latency event gets out the issue scheduling queue and gets back on its long event completion. In this work, for simplicity we use long latency event as IQ flush on a branch misprediction, we can easily extend it to other long event kinds. Fig. 7 demonstrates four threads of 11 instructions each under an assumption that threads 0,1,2 have data dependence between instructions I3, I4. An efficient issue scheduling algorithm enables maximum efficiency of a 2-way pipeline for issuing two instructions in parallel on each cycle. In SMT example there is only one idle slot compared to three in CGMT, four in FGMT, and five in ST. Recall that data and control dependences between two adjacent instructions of the same thread or structural dependence between two adjacent instructions of different threads cause an idle slot.

B. RRP – THE PROPOSED ISSUE SCHEDULING ALGORITHM

1) INTRODUCTION

This section introduces the proposed issue scheduling SMT algorithm named Round-Robin in Pairs (RRP) and its implementation. Recall that the issue scheduling algorithm in a 2-Way processor is responsible for providing a continuous stream of instruction pairs under the consideration of structural, data, and control dependencies. As aforementioned, in the case of a 2-Way in-order processor, the input of the issue scheduling algorithm has size of 2×4 instructions, i.e., eight instructions. The total possible permutations for choosing four instruction pairs out of the four instruction queues (each IQ issues an instruction pair) is equal to 52 [31]. Due to the large number of permutations, and since the permutations number grows with the number of hardware threads, an optimal issuing algorithm that supports full permutation coverage is infeasible. The proposed RRP algorithm is based on a simple permutation selection approach. To simplify the permutation options, we define a group of threads, so-called coupled threads, from which an instruction pair can be issued. The algorithm aims to continuously issue two instructions from two different threads (out of the four IQs) using a round-robin (RR) approach. Since each instruction pair is composed of two instructions issued from different threads, the data dependency within the issued pair is canceled, and therefore a full TLP can be achieved. However, a DATA dependency can still occur between two instructions belonging to the same thread and are piped sequentially. In case the instruction execution time takes more than one cycle (for example MUL and LOAD require two cycles) a single STALL is required. Also, structural and control dependencies can arise. For example, when two instructions from two different threads require the same system resource but there is only a single resource (such as for MUL or LSU type commands) then there is a structural dependency that requires a penalty of a single stall. A control type dependency can arise when there is a branch prediction miss that causes the queue to be flushed and a thread switch is made to another thread that requires a penalty of a single cycle.

2) PAIRS PARTITIONING

The RRP algorithm is based on determining predefined threads partitioning into groups of coupled threads. In the case of four threads, two threads are assigned to a predefined group and are considered coupled threads.

Fig. 8. depicts the *IIS-BUF* composed of 8 issued instructions (each row represents 2 instructions from the same thread). The figure shows three options for thread group partitioning, determining three selected permutations (out of the total of 52 possible permutations), each cover four instruction pair. For example, Fig 8(a) depicts two possible groups, threads (0-1) and threads (2-3), meaning that an instruction pair can be composed only of instructions derived from the same group. The permutation depicted in Fig.8(a) covers 4 instruction pairs: (0,2), (1,3), (4,6), and (5,7). The proposed RRP algorithm is based on assigning each two threads to a common group, by choosing one of the three options depicted in Fig. 8. For simplicity, in the rest of the algorithm description we use the permutation depicted in Fig. 8(a). In this case, threads TD0 and TD1 are chosen as coupled threads (i.e., belong to the same group), and TD2 and TD3 are also coupled threads.

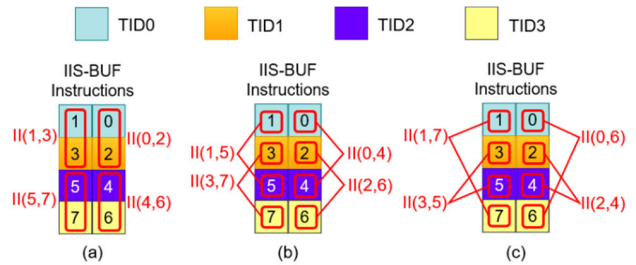


FIGURE 8. Three options of pairs partitioning definition where each option is based on two groups of coupled threads. Each instructions pair is selected from a coupled thread. (a) Threads 0, 1 are coupled and threads 2, 3 are coupled. (b) Threads 0, 2 are coupled and threads 1, 3 are coupled (c) Threads 0, 3 are coupled and threads 1, 2 are coupled.

3) THE ALGORITHM

This section describes the proposed RRP algorithm in detail. The description is accompanied by an example based on predefined thread pair partitioning, as shown in Fig.8 (a). The RRP algorithm is based on RR among groups of threads where each cycle we issue instructions pair from a group. Fig.9 depicts eleven main cases of instructions pair issue scheduling which happen following series of scenarios: (a) the algorithm works without any constraint except of In-Order issuing in a thread, so the stream of instructions pairs is $\{II(0,2), II(4,6), II(1,3), II(5,7)\}$ circularly. (b) There is a long event occurrence which leads to IQ-flush associated with an instruction from TID0 or TID1. In this case, the group1 that contains threads TID0 and TID1 is deactivated (until a long event ends). Fig.9(b) depicts an instance where instruction I2 of TID1 has caused IQ1-flush and hence group1 is deactivated, therefore the algorithm issues instructions pairs only from group2 $\{II(4,6), II(5,7)\}$ circularly. (c) This scenario happens at the end of the scenario depicted in Fig.9(b)

and after group1 is reactivated. The instruction pair to be issued from group1 is I1 of TID0 and I2 of TID1 due to the IQ1-flush, so the instructions pairs of group1 are flipped from $\{II(0,2), II(1,3)\}$ to $\{II(1,2), II(0,3)\}$. Therefore, the continuous stream of instructions pairs of this case is $\{II(0,3), II(4,6), II(1,2), II(5,7)\}$ circularly, as shown in Fig.9(c). (d) This scenario is the symmetrical case of Fig.9(b) when there is a long event occurrence which leads to IQ-flush associated with an instruction from TID2 or TID3 where group2 is deactivated. Therefore, the continuous stream of instructions pairs of this case is $\{II(0,2), II(1,3)\}$ circularly. (e) This scenario is the symmetrical case of Fig.9(c) when group2 is reactivated. The instruction pair to be issued from group2 is I5 of TID2 and I6 of TID3 due to the IQ3-flush, so the instructions pairs of group2 are flipped from $\{II(4,6), II(5,7)\}$ to $\{II(5,6), II(4,7)\}$. Therefore, the continuous stream of instructions pairs of this case is $\{II(0,2), II(4,7), II(1,3), II(5,6)\}$ circularly. (f) This scenario depicts an inclusion of cases Fig.9(b) and Fig.9(d) where there are long events occurrence which leads to IQ-flush associated with two instructions, one from TID0 or TID1 and one from TID2 or TID3. Because we cannot deactivate all groups, i.e., group1 and group2 the continuous stream of instructions is from the recently issued group. Fig.9(f) depicts an instance where instruction I1 of TID0 has caused IQ0-flush and instruction I4 of TID2 has caused IQ2-flush. In case the recently issued group is group1, the continuous stream of instructions pairs is $\{II(2,3)\}$ circularly, encircled in red. In case the recently issued group is group2, the continuous stream of instructions pairs is $\{II(6,7)\}$ circularly, encircled in blue. (g) This scenario depicts an inclusion of cases Fig.9(c) and Fig.9(e). (h) This case is the same as case Fig.9(b) except that group2 instructions pairs are flipped. (i) This case is the same as case Fig.9(b) except that group1 instructions pairs are flipped. (j) This scenario is the symmetrical case of Fig.9(f) where instruction I3 of TID1 has caused IQ1-flush and instruction I6 of TID3 has caused IQ3-flush. (k) The case threads finish their code running in different order (Out-of-Order execution among threads) which causes them to become invalid in different order. Fig.9(k) depicts an instance where the TID0 and TID3 have finished their code running, TID1 remains last six instructions, and TID2 remains last four instructions. Therefore, the series to be issued is $\{II(2,3), II(4,5), II(2,3), II(4,5), II(2,3)\}$ and after that all four threads are invalid. The above eleven cases can also describe five chronological orders of scenarios: (1) $a \leftrightarrow b \leftrightarrow c$, (2) $a \leftrightarrow d \leftrightarrow e$, (3) $a \leftrightarrow \text{forj} \leftrightarrow g$, (4) $g \leftrightarrow h \leftrightarrow e$, (5) $g \leftrightarrow i \leftrightarrow c$.

C. RRP – HARDWARE IMPLEMENTATION

1) RRP – ISSUE SCHEDULER MODULE

Fig.10 depicts the RRP HW module which includes:

module inputs:

- (i) Eight inputs II (0)-II (7) from IIS-BUF (two instructions from each TID). Each II(i) entry has width of 68 bits which contained from a single 32-bit instruction word,

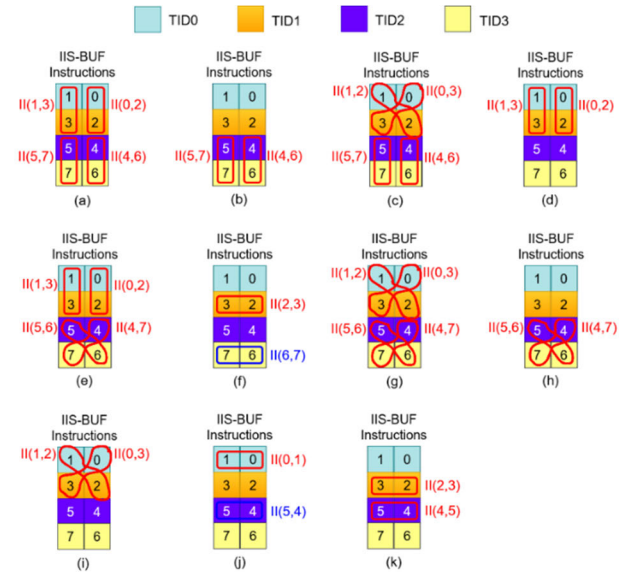


FIGURE 9. Eleven main cases (a)-(k) of RRP algorithm.

32-bit Thread PC, 2-bit Thread TID and two System Status bits.

- (ii) A valid bit is assigned to each II(i), valid [7:0].
 - (iii) Four status bits for each TID: valid, empty, long event, flush occurrence.
 - (iv) Two complete bits are assigned to each TID IQ outputs
- module outputs:*
- (v) Two outputs for the two issued instructions (one for each pipeline channel), and their associated thread ID.
 - (vi) An additional two pop bits indicate that the popped instructions can be issued.

2) RRP – HW ARCHITECTURE

Fig.11 shows the Mealy FSM diagram for the RRP hardware implementation. The FSM contains four main states (S0-S3). As explained in Sec. II, there are two coupled threads in each defined group and a total of four coupled threads (for the two groups). Therefore, the state machine includes four states, each representing a specific coupled thread. In each state (i.e., coupled thread) there are five different options to select an instruction pair. Each state has two inputs and two outputs. A state transition occurs while a new instruction pair should be issued according to the RRP algorithm, as depicted by the transition equations. The various scenarios depicted in Fig. 9 are fully represented by the state transitions, and according to the state transition equations (Fig. 11).

State transition conditions:

There are five auxiliary equations for state transition:

- (i) TID_i_ok = indicates whether a thread TID_i is valid **and** has no event occurrence **and** its IQ_i is not empty.
- (ii) $group1_cond0$ = all threads of group1 are OK (i.e., $TID0_ok = 1$ **and** thread $TID1_ok = 1$).
- (iii) $group1_cond1$ = indicates that only one thread of group1 is OK **and** the other is not valid.

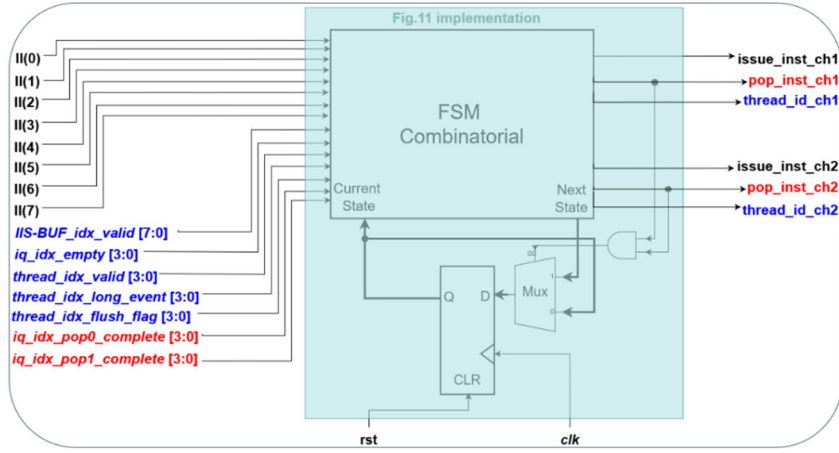


FIGURE 10. RRP issue scheduler module.

- (iv) group2_cond0 = all threads of group2 are OK (i.e., TID2_ok = 1 **and** thread TID3_ok = 1).
- (v) group2_cond1 = indicates that only one thread of group2 is OK **and** the other is not valid.

State transition equations:

There are two transition equations of the Mealy FSM:

- (i) group1_trans = group1_cond0 || group1_cond1
- (ii) group2_trans = group2_cond0 || group2_cond1

State output conditions:

There are five state output conditions of the Mealy FSM:

- (i) flush_TID_i = indicates whether a thread TID_i had odd number of times of flush occurrence
- (ii) group1_TID0_valid = indicates that only the thread TID0 of group1 is valid, that is, TID0_ok = 1 **and** TID1_ok = 0) i.e., TID1 is invalid **or** has event occurrence)
- (iii) group1_TID1_valid = indicates that only the thread TID1 of group1 is valid, that is, TID1_ok = 1 **and** TID0_ok = 0) i.e., TID0 is invalid **or** has event occurrence)
- (iv) group2_TID2_valid = indicates that only the thread TID2 of group2 is valid, that is, TID2_ok = 1 **and** TID3_ok = 0) i.e., TID3 is invalid **or** has event occurrence).
- (v) group2_TID3_valid = indicates that only the thread TID3 of group2 is valid, that is, TID3_ok = 1 **and** TID2_ok = 0) i.e., TID2 is invalid **or** has event occurrence).

The matching between the cases that given in Fig. 9 and the FSM Mealy of the RRP issue scheduler in Fig. 11, can be seen using the next examples: (1) scenario Fig. 9(a) is equivalent to the circular states transition $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3$ with the transition signals $group1_trans = 1$, $group2_trans = 1$, and all the rest signals have value of zero. (2) scenario Fig. 9(b) is equivalent to the circular states transition $S1 \rightarrow S3$ with the transition signals $group2_trans = 1$, and all the rest signals have value of zero. (3) scenario Fig. 9(c) is equivalent to the

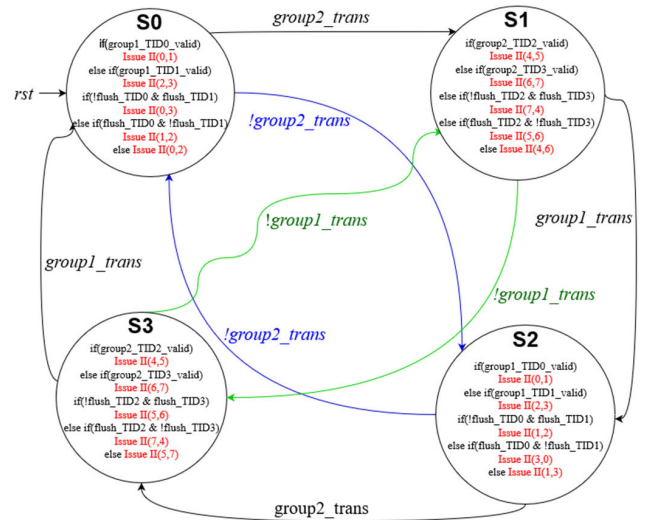


FIGURE 11. FSM Mealy diagram of the RRP issue scheduler.

circular states transition $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3$ with the transition signals $group1_trans = 1$, $group2_trans = 1$, $flush_TID1 = 1$, and all the rest signals have value of zero. (4) scenario Fig. 9(f) has two options. It is equivalent to the circular states transition $S0 \rightarrow S2$ when the start state is $S0$ or $S2$ with the transition signals $group1_TID1_valid = 1$, and all the rest signals have value of zero. Option 2, it is equivalent to the circular states transition $S1 \rightarrow S3$ when the start state is $S1$ or $S3$ with the transition signals $group2_TID3_valid = 1$, and all the rest signals have a value of zero. (5) scenario Fig. 9(g) is equivalent to the circular states transition $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3$ with the transition signals $group1_trans = 1$, $group2_trans = 1$, $flush_TID0 = 1$, $flush_TID2 = 1$ and all the rest signals have value of zero. (6) scenario Fig. 9(k) is equivalent to the five states transition $S0 \rightarrow S1 \rightarrow S2 \rightarrow S3 \rightarrow S1$ with the transition signals $group1_trans = 1$, $group2_trans = 1$, $flush_TID0 = 1$, $group1_TID1_valid = 1$, $group2_TID2_valid = 1$, and all the rest signals have value of zero.

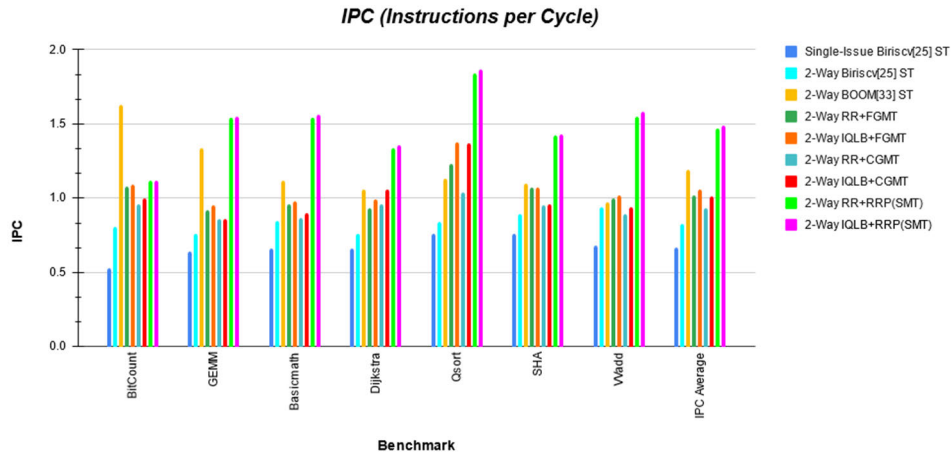


FIGURE 12. IPC of the various architectures for the seven benchmarks.

VI. EXPERIMENTAL RESULTS

This section describes the performance evaluation of the proposed SUMT-V architecture. To evaluate the efficiency of the suggested IQLB prefetch scheduling and the RRP issue scheduling algorithms we tested different architectures and various scheduling algorithms, as depicted in Fig. 12. The following three ST-based architecture were examined: (a) single-issue ST Biriscv core [24], (b) 2-way ST Biriscv core [24], and (c) 2-way ST out-of-order with branch prediction support BOOM core [32]. The proposed SUMT-V architecture was tested with various scheduling algorithms: (a) IQLB and RR with FGMT, (b) IQLB and RR with CGMT, and (c) IQLB and RR with RRP (SMT). The evaluation was carried out in terms of power, performance, and area (PPA), using a suite of seven benchmark applications (GEMM, Vector-vector addition, Basicmath, Dijkstra, Qsort, Bit Count, and SHA) derived from the common Mi-Bench [16] and RISCv benchmark [17].

A. PERFORMANCE EVALUATION

Fig.12 shows the performance results in IPC terms (i.e., number of instructions per cycle) for the various examined architectures. The IQLB and RRP configuration demonstrates the best performance for all the benchmarks excluding the BitCount. A deep inspection of the BitCount code shows that most of the code contains conditional branches, and therefore, the Boom architecture that supports branch prediction outperforms all the others. The main contribution to the improvement is due to the proposed RRP.

Fig. 13 shows the speed-up achieved with the proposed RRP scheduling algorithm compared to the two 2-way ST-based architectures (i.e., the Biriscv and BOOM [24], [32]), and compared to the two references scheduling algorithms (i.e., the FGMT, and CGMT), both applied to the 2-way SUMT-V MT-based architecture. The speedup of μArc1 compared to μArc2 in terms of IPC is defined as follows: $\frac{IPC_{\mu\text{Arc1}} - IPC_{\mu\text{Arc2}}}{IPC_{\mu\text{Arc2}}} \cdot 100[\%]$. The average speed-up

in terms of IPC is about 50% across all benchmarks. The MT-based architecture with RRP outperforms the ST Biriscv architecture, demonstrating (as expected) a significant speedup of 79.5%, while a speedup of only 23.2% is achieved compared to the ST BOOM. This can be explained since the latest is an out-of-order architecture and supports branch prediction.

The speedup achieved for the CGMT (56.5%) is greater than that of the FGMT (42.9%) which can be explained since the CGMT is more sensitive to data dependency and therefore the potential for improvement is higher.

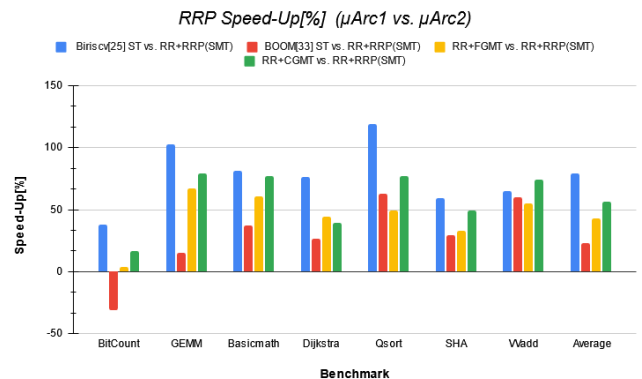


FIGURE 13. RRP speed-up compared to Biriscv and BOOM (2-way ST) and to the FGMT and CGMT scheduling (2-way MT).

For the ST-based Biriscv architecture, a 100% and 120% speedup are demonstrated for the GEMM and the QSORT applications, respectively. While, compared to the BOOM architecture the speedup decreased to about 15% and 63%, respectively. As previously explained, the BOOM architecture outperforms the proposed SUMT-V for the BitCount application. Therefore, this is the only scenario a negative speedup is demonstrated, as depicted in Fig. 13.

Fig. 14 shows the performance of the IQLB prefetch scheduling for the same benchmark compared to the RR

algorithm, and for the three scheduling algorithms: the proposed RRP, FGMT and CGMT. The average speed-up in terms of IPC, due to the contribution of the proposed IQLB prefetch algorithm alone, is about 4.5% across all benchmarks. The best performance is demonstrated by the combination of the IQLB with the CGMT issue scheduling for most of the applications, showing a speedup of 10.4% and 31.7% for Dijkstra and QSORT, respectively. For the FGMT a speedup of 6.4% and 12.2% is achieved for the Dijkstra and QSORT, respectively.

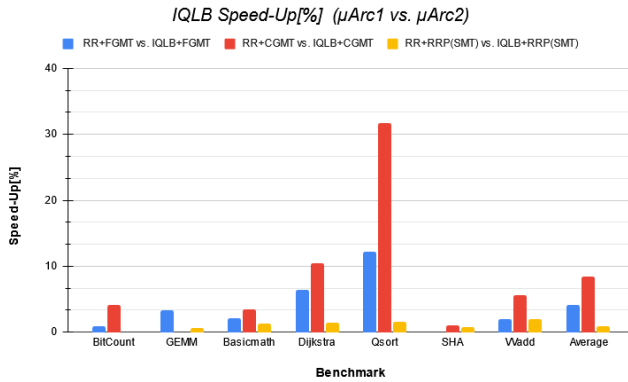


FIGURE 14. IQLB speed-up compared to RR for the three issue-scheduling algorithms: RRP, FGMT and CGMT.

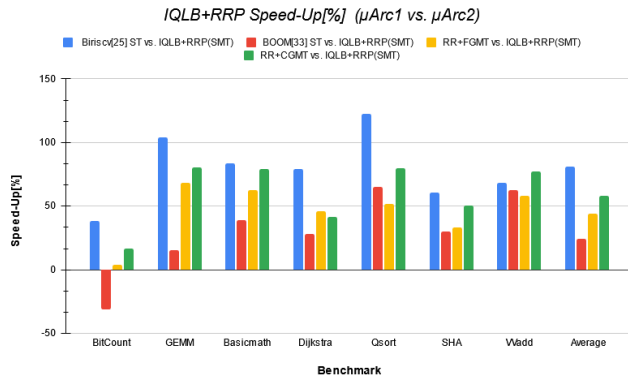


FIGURE 15. IQLB+RRP speed-up compared to 2-way ST, RR+FGMT and RR+CGMT scheduling (2-way MT).

B. PPA AND ENERGY EFFICIENCY ANALYSIS

The proposed SUMT-V architecture was evaluated and compared in terms of PPA to 2-way ST-based Biriscv [24] baseline for various combinations of prefetch and issue scheduling algorithms. The SUMT-V architecture was fully implemented in Verilog using Cyclone IV FPGA chip (specifically the Cyclone IV E EP4CE115F29C7 integrated in DE2-115 board) [33]. A functional verification was carried out via extensive RTL simulations using Intel ModelSim tool with the use of testbenches. A system-level verification was performed via on chip capabilities, the Signal-Tap embedded logic analyzer and the in-system memory JTAG-based access. The Intel Quartus Prime tool [34] has been used for

system-level verification, synthesis purposes, and for power and area evaluation.

The energy efficiency of the proposed architecture has been analyzed to demonstrate an improvement in overall efficiency. Energy per instruction (EPI) [35] measures power efficiency by recording the average amount of energy expended per instruction executed by the CPU. EPI is measured in [Joules/instruction] and is defined as follows:

$$EPI = \frac{\text{Joules}}{\text{Instruction}} = \frac{\frac{\text{Joules}}{\text{Second}}}{\frac{\text{Instruction}}{\text{Second}}} = \frac{\text{Watt}}{IPS} = \frac{\text{Watt}}{IPC \cdot f_{MCLK}} \left[\frac{\text{Joule}}{\text{Instruction}} \right] \quad (1)$$

Table 2 summarizes the synthesis results for the various SUMT-V based architectures and the ST baseline RISC-V in terms of PPA and Energy Efficiency. The implementation of the IQLB&RRP requires 26,185 logic elements (LE) and 8062 registers and consumes on average 261.61 mW. The best performance is achieved while using the RRP as the issue-scheduling algorithm. The IQLB&RRP and RR&RRP combinations outperform all other configurations, demonstrating an average speed-up of about 52.1% in terms of IPC (as detailed in Sec VI-A). The performance improvement is achieved at the cost of about a 10mW increase in power and an additional of 77 registers. The IQLB&RRP and RR&RRP combinations outperform all other configurations in terms of EPI, demonstrating an average EPI of 3.525 [nJ / Instruction]. The best energy efficiency is achieved while using the proposed RRP as the issue-scheduling algorithm.

TABLE 2. PPA and energy-efficiency results for various SUMT-V based architectures (implemented on the Cyclone IV FPGA).

Micro-Architecture Design @ 50MHz	Area		Power	Performance	Energy efficiency
	LEs (Logic Elements)	Registers	Power Average [mW]	IPC Average	EPI[35] Average [nJ / Instruction]
Biriscv[24] Single-Issue ST	9,498	3,441	237.99	0.67	7.104
Biriscv[24] 2-Way ST	11,345	3,559	248.15	0.83	5.979
RR+FGMT	26,945	7985	250.75	1.02	4.916
IQLB+FGMT	27,162	7985	252.14	1.06	4.757
RR+CGMT	27,160	7985	253.77	0.93	5.457
IQLB+CGMT	27,184	7985	259.81	1.01	5.144
RR+RRP(SMT)	26,095	8062	260.49	1.47	3.544
IQLB+RRP(SMT)	26,185	8062	261.61	1.49	3.511

The power average was estimated using the Intel Quartus Power Analyzer (QPA) tool. The achievable maximum frequencies (f-max) for the Biriscv Single-Issue ST and Biriscv 2-Way [24] are 80.35MHz and 77.78MHz, respectively, while for our various SUMT-V architecture the f-max is in the range of 75 to 77.75 MHz. The 50 MHz frequency is chosen as a round frequency for comparison purposes.

Table 3 depicts the additional cost required for implementing the proposed SUMT-V based architectures compared to

TABLE 3. SUMT-V architectures vs. the 2-way ST BIRISCV [24] in terms of area and power cost.

Micro-Architecture Design @ 50MHz	Additional Area[%]		Average of Additional Power Dissipation [%]	Average of IPC Speed-Up [%]
	Additional LEs [%]	Additional Registers [%]		
RR+FGMT	137.5	124.36	1.04	22.89
IQLB+FGMT	139.41	124.36	1.6	27.71
RR+CGMT	139.4	124.36	2.26	12.04
IQLB+CGMT	139.61	124.36	4.69	21.68
RR+RRP(SMT)	130.01	126.52	4.97	77.1
IQLB+RRP(SMT)	130.8	126.52	5.42	79.51

the 2-way ST-based BIRISCV baseline in terms of area and power. The achieved IPC speed up (ranges from 12% to 80%) is achieved in average cost of additional 136% LEs and 125% registers. The power consumption is increased of about 14mW.

VII. SUMMARIZE AND CONCLUSION

This work presents a new MT-based micro-architecture named SUMT-V. The proposed architecture is composed of a 2-way superscalar in-order Simultaneous multithreading (SMT) applied to a superscalar RISC-V processor. This work also proposes new efficient prefetch scheduling (IQLB) and Issue-scheduling (RRP) algorithms for Simultaneous Multithreading processors. The proposed scheduling approaches utilize the unique RISC-V ISA property, enabling decoding an instruction type in the early pipeline stage. Results show that the proposed prefetch and issue scheduling algorithms outperform the 2-way superscalar Fine-Grained Multi-Threaded (FGMT) and the Coarse-Grained Multi-Threaded (CGMT) approaches.

Combining the IQLB and RRP(SMT) is a cost-effective approach. The performance improvement is significant, while the hardware cost is negligible relative to RR+FGMT and RR+CGMT-based cores. An average core utilization improvement of up to about 52% in terms of IPC using four threads is demonstrated using common benchmarks. The suggested micro-architecture was fully implemented in Verilog and implemented on an FPGA chip.

The outcome of this research may contribute to the throughput improvement of IoT edge devices and RISC-V-based embedded processors. Future work may improve performance by utilizing various thread group permutations to avoid structural dependency and integrating dynamic branch prediction approaches.

REFERENCES

- [1] C. Celio, D. A. Patterson, and K. Asanović, "The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep., 2015.
- [2] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, and K. Asanović, "BOOMv2: An open-source out-of-order RISC-V core," in *Proc. 1st Workshop Comput. Archit. Res. RISC-V (CARRV)*, 2017, pp. 1–12.
- [3] G. Chrysos, "Intel® Xeon Phi™ coprocessor—The architecture," *Intel Whitepaper*, vol. 176, p. 43, Mar. 2014.
- [4] H. Q. Le, J. A. Van Norstrand, B. W. Thompto, J. E. Moreira, D. Q. Nguyen, D. Hruscky, M. J. Genden, and M. Kroener, "IBM POWER9 processor core," *IBM J. Res. Develop.*, vol. 62, nos. 4–5, pp. 1–12, Jul. 2018.
- [5] J. L. Shin, K. Tam, D. Huang, B. Petrick, H. Pham, C. Hwang, H. Li, A. Smith, T. Johnson, F. Schumacher, D. Greenhill, A. S. Leon, and A. Strong, "A 40nm 16-core 128-thread CMT SPARC SoC processor," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2010, pp. 98–99.
- [6] S. Hily and A. Seznec, "Out-of-order execution may not be cost-effective on processors featuring simultaneous multithreading," Inria, Le Chesnay-Rocquencourt, France, Tech. Rep. RR-3391, 1998.
- [7] S. K. Sadasivam, B. W. Thompto, R. Kalla, and W. J. Starke, "IBM Power9 processor architecture," *IEEE Micro*, vol. 37, no. 2, pp. 40–51, Mar. 2017.
- [8] B. W. Curran, C. Jacobi, J. J. Bonanno, D. A. Schroter, K. J. Alexander, A. Puranik, and M. M. Helms, "The IBM z13 multithreaded microprocessor," *IBM J. Res. Develop.*, vol. 59, nos. 4–5, pp. 1–13, Jul. 2015.
- [9] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Amsterdam, The Netherlands: Elsevier, 2019.
- [10] C. Bechara, A. Berhault, N. Ventroux, S. Chevobbe, Y. Lhuillier, R. David, and D. Etiemble, "A small footprint interleaved multithreaded processor for embedded systems," in *Proc. 18th IEEE Int. Conf. Electron., Circuits, Syst.*, Dec. 2011, pp. 685–690.
- [11] M. Howard and A. Kopsner, "Design of the tera MTA integrated circuits," in *GaAs IC Symp. IEEE Gallium Arsenide Integr. Circuit Symp. 19th Annu. Tech. Dig.*, Apr. 1997, pp. 14–17.
- [12] A. Agarwal, B.-H. Lim, D. Kranz, and J. Kubiatowicz, "APRIL: A processor architecture for multiprocessing," in *Proc. 17th Annu. Int. Symp. Comput. Archit.*, 1990, pp. 104–114.
- [13] A. Cheikh, S. Sordillo, A. Mastrandrea, F. Menichelli, and M. Olivieri, "Efficient mathematical accelerator design coupled with an interleaved multi-threading RISC-V microprocessor," in *Proc. Int. Conf. Appl. Electron. Pervading Ind., Environ. Soc. Cham, Switzerland: Springer*, Jan. 2019, pp. 529–539.
- [14] S. Kvatinsky, Y. H. Nacson, Y. Etsion, E. G. Friedman, A. Kolodny, and U. C. Weiser, "Memristor-based multithreading," *IEEE Comput. Archit. Lett.*, vol. 13, no. 1, pp. 41–44, Jan. 2014.
- [15] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: Maximizing on-chip parallelism," in *Proc. 22nd Annu. Int. Symp. Comput. Archit.*, vol. 995, 1995, pp. 392–403.
- [16] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. 4th Annu. IEEE Int. Workshop Workload Characterization*, Austin, TX, USA, Mar. 2001, pp. 3–14, doi: 10.1109/WWC.2001.15.
- [17] *RISC-V Benchmarks*. Accessed: Feb. 20, 2023. [Online]. Available: <https://github.com/riscv/riscv-tests/tree/master/benchmarks>
- [18] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*, document Version 20191213, RISC-V Found., 2019.
- [19] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, and A. Izraelevitz, "The rocket chip generator," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2016-17, 2016.
- [20] F. Conti, D. Rossi, A. Pullini, I. Loi, and L. Benini, "Energy-efficient vision on the PULP platform for ultra-low power parallel computing," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2014, pp. 1–6.
- [21] P. D. Schiavone, F. Conti, D. Rossi, M. Gautschi, A. Pullini, E. Flamand, and L. Benini, "Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications," in *Proc. 27th Int. Symp. Power Timing Model., Optim. Simulation (PATMOS)*, Sep. 2017, pp. 1–8.
- [22] K. Patsidis, D. Konstantinou, C. Nicopoulos, and G. Dimitrakopoulos, "A low-cost synthesizable RISC-V dual-issue processor core leveraging the compressed instruction set extension," *Microprocessors Microsys.*, vol. 61, pp. 1–10, Sep. 2018.
- [23] MIT CSAIL's Comput. Struct. Group. (2017). *Riscy Processors—Open-Sourced RISC-V Processors*. [Online]. Available: <https://www.github.com/csail-csg/riscy>
- [24] Ultraembedded Repository RISC-V CPU. (2021). *RISC-V Processors—Open-Sourced*. [Online]. Available: <https://github.com/ultraembedded>

- [25] C. Collange, "Simty: Generalized SIMT execution on RISC-V," in *Proc. 1st Workshop Comput. Archit. Res. RISC-V (CARRV)*, Oct. 2017, p. 6.
- [26] A. Cheikh, G. Cerutti, A. Mastrandrea, F. Menichelli, and M. Olivieri, "The microarchitecture of a multi-threaded RISC-V compliant processing core family for IoT end-nodes," in *Proc. Int. Conf. Appl. Electron. Pervading Ind., Environ. Soc.* Cham, Switzerland: Springer, Jul. 2017, pp. 89–97.
- [27] J. M. Escobar, "A multithreading RISC-V implementation for Lagarto architecture," Master's thesis, Dept. Comput. Archit., Universitat Politècnica de Catalunya, Vilanova i la Geltrú, Spain, 2020.
- [28] Y. Eni, S. Greenberg, and Y. Ben-Shimol, "Efficient hint-based event (EHE) issue scheduling for hardware multithreaded RISC-V pipeline," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 735–745, Feb. 2022, doi: [10.1109/TCSI.2021.3117490](https://doi.org/10.1109/TCSI.2021.3117490).
- [29] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V instruction set manual, volume I: User-level ISA, version 2.1," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2016-118, 2016.
- [30] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanovic, "The RISC-V instruction set manual volume II: Privileged architecture version 1.9," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2016-129, 2016.
- [31] H. Ribo, *A Combinatorial Evaluation of Instruction Pairs Coverage Permutations in an Instruction Image of Size 2x4*. Accessed: Dec. 28, 2023. [Online]. Available: <https://github.com/RiboHan/SUMT-V>
- [32] A. A. Owahid and E. B. John, "Wasted dynamic power and correlation to instruction set architecture for CPU throttling," *J. Supercomput.*, vol. 75, no. 5, pp. 2436–2454, May 2019, doi: [10.1007/s11227-018-2637-6](https://doi.org/10.1007/s11227-018-2637-6).
- [33] TerasIC. *Altera DE2-115 Development and Education Board*. Accessed: Mar. 10, 2023. [Online]. Available: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=502>
- [34] Intel®. *Quartus® Prime Design Software*. Accessed: Mar. 10, 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html>
- [35] E. Grochowski and M. Annavaram, "Energy per instruction trends in Intel microprocessors," *Technol. Intel Mag.*, vol. 4, no. 3, pp. 1–8, 2006.



HANANYA RIBO received the B.Sc. degree (Hons.) in electrical and computer engineering from the Ben-Gurion University of the Negev, Beer-Sheva, Israel, in 2010. His research interests include computer architecture, RT embedded systems, real-time/low-power processor architectures, and hardware accelerators architecture.



SHLOMO GREENBERG (Member, IEEE) received the B.Sc., M.Sc. (Hons.), and Ph.D. degrees in electrical and computer engineering from the Ben-Gurion University of the Negev, Beer-Sheva, Israel, in 1976, 1984, and 1997, respectively. He is currently an Associate Professor and the Head of the Computer Science Department, Sami Shamoon College of Engineering, and a Staff Member with the School of Electrical and Computer Engineering, Ben-Gurion University of the Negev. His main research interests include computer architecture, machine learning, image and digital signal processing, computer vision, and VLSI low-power design.

...